



Investigación sobre herramientas de desarrollo web en entorno servidor

Para esta tarea se ha elegido FastAPI, que es un framework de Python.

¿Qué es FastAPI?

- Es un framework de Python que sirve principalmente para crear APIs y aplicaciones web.
- Está basado en Starlette (kit de herramientas, diseñado para construir servicios y aplicaciones web asíncronas) y Pydantic (librería de validación de datos)
- Genera documentación automática.

Requisitos para el desarrollo y ejecución de las páginas web:

- Se requiere Python 3.8 o superior.
- Tener instalado FastAPI (obviamente)
- Un entorno virtual de Python
- Uvicorn, que actúa como un servidor ASGI (Asynchronous Server Gateway Interface), que permite ejecutar aplicaciones FastAPI y recibir peticiones HTTP
- Editor o IDE

Se instala poniendo en la terminal del entorno **pip install "fastapi[standard]"**.

La app se ejecuta poniendo en la terminal del entorno `uvicorn main:app --reload` y luego en local escucha desde el puerto 8000

Compatibilidad con los servidores web

FastAPI utiliza el estándar ASGI, por lo que necesita un servidor compatible como Uvicorn. Puede ejecutarse detrás de los servidores web de Nginx o Apache y es compatible con Docker y los servicios de alojamiento en la nube. Funciona en Windows, macOS y Linux.

¿Qué IDEs se usan con FastAPI?

Se usa Visual Studio Code, ya que es ligero y con muy buen soporte para Python y también se usa PyCharm por su entorno profesional.

Ejemplos de páginas o programas realizados con FastAPI

Se utiliza principalmente para crear APIs y aplicaciones web. Algunos ejemplos son:

- APIs REST para aplicaciones móviles o páginas web (REST es el conjunto de reglas/conceptos para diseñar APIs utilizando HTTP (GET,POST,PUT,DELETE))
- Sistemas de gestión de usuarios, productos o pedidos.
- Aplicaciones web con HTML, CSS y JavaScript
- Servicios de IA y machine learning que ofrecen modelos mediante una API.
- Backends para apps que necesitan comunicarse con una base de datos.

Algunos ejemplos reales que usan FastAPI son Netflix, para proyectos relacionados con sus servicios internos, también Uber para proyectos de machine learning como servicios para realizar predicciones.

Diferencias de otros frameworks

FastAPI está pensado para realizar el backend y las APIs de forma rápida con validación de datos y documentación automática, mientras otros como Flask son más minimalistas y tu puedes elegir que tipo de componentes quieres, Django es mucho más completo ya que proporciona muchas cosas ya preparadas como administración, autenticación y estructura del proyecto. Saliendo de Python, frameworks como [Node.js](#) permiten crear servidores y APIs utilizando JavaScript, una opción buena cuando se quiere utilizar JavaScript tanto en Frontend como en Backend.

Prueba de Concepto

Hemos definido el siguiente código en [main.py](#)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def inicio():
    return {"mensaje": "API funcionando correctamente"}

@app.get("/usuario/{nombre}")
def obtener_usuario(nombre: str):
    return {
        "nombre": nombre,
        "mensaje": "Usuario encontrado"
    }

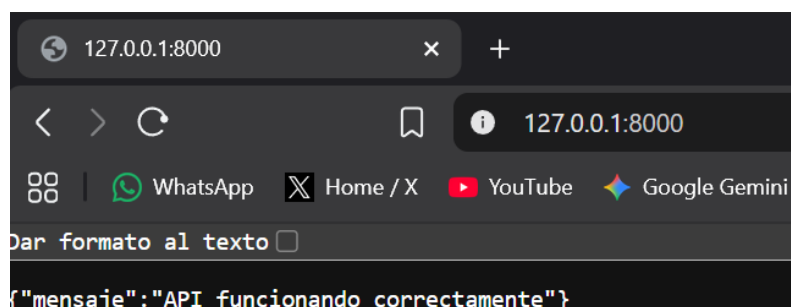
@app.get("/holaquetal")
def obtener_holaquetal():
    return {
        "esto es fastapi" : "ok"
    }

def main(): 1 usage
    uvicorn.run(app, host="localhost", port=8000)

main()
```

Vemos que se ejecuta correctamente y atiende a las peticiones:

```
INFO: Started server process [13836]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://localhost:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:51593 - "GET /holaquetal HTTP/1.1" 200 OK
INFO: 127.0.0.1:51593 - "GET /holaquetal HTTP/1.1" 200 OK
INFO: 127.0.0.1:51593 - "GET /holaquetal HTTP/1.1" 200 OK
INFO: 127.0.0.1:51593 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:51599 - "GET /usuario/Carlos HTTP/1.1" 200 OK
```



The screenshot shows a web browser window with the address bar set to 127.0.0.1:8000. The browser's developer tools are open, displaying the response of a GET request to the root endpoint. The response is a JSON object: {"mensaje": "API funcionando correctamente"}. The browser's address bar shows the URL 127.0.0.1:8000. The browser's tabs show 127.0.0.1:8000. The browser's bookmarks bar shows icons for WhatsApp, Home / X, YouTube, and Google Gemini. The browser's search bar shows the text "Dar formato al texto".